Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

[acá va tu titulo]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: [acá va tu director]

Codirector: [acá va tu codirector]

Buenos Aires, 2026

[ACÁ VA TU TITULO]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[acá van los agradecimientos]

Índice general

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ. It has only one tape from where it reads the input and in an autoregressive way writes in it more tokens. There are two distinguished tokens called stopping symbols. Once the transformer writes one of those it halts. We say that a transformer accepts a word if when it halts the stopping symbol printed is the accepting one. Correspondingly, a transformer rejects a word if the stopping symbol printed is the rejecting one.

In each step the transformer maps the content of the tape to a probability distribution over the alphabet. Depending on this distribution the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called chain of thought.

Define n_{max} an say smt about s of the tape

The process of generating the next token consists of two separated steps. First a probability distribution over the alphabet for the token is computed by a process called distribution generation. After that, it is necessary to decide the specific token by a process called token selection. These two algorithms together, with their parameters, is what defines the transformer. During this work we will analyze and compare two different options for token selection.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ. This process consists of four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT), and a stack of L identical layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network (FF). Each part and layer has its own trainable parameters. Then we can define the transformer by its parameters $\theta = \{\theta_{\text{TE}}, \theta_{\text{PE}}, \{\theta^l_{\text{ATTN}}, \theta^l_{\text{FF}}\}_{l=0}^{L-1}, \theta_{\text{OUTPUT}}\}$.

Token Embedding: It's a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\text{TE}} \in \mathbb{R}^{d\times|\Sigma|}$. For all $a \in \Sigma$, we note $\theta_{\text{TE}}(a) \in \mathbb{R}^d$ to the mapping of token a.

Position Encoding: It's a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\text{PE}} \in \mathbb{R}^{d\times n_{max}}$. For all position of the tape $1 < i < n_{max}$ we note $\theta_{\text{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i.

Self Attention Layer: It's a mapping from $(\mathbb{R}^d)^n$ to $(\mathbb{R}^d)^n$. Given parameters $\theta_{\text{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d$, the layer it's defined by the algorithm 1. This layer is defined only as a single head attention layer because we can simulate 1 multi-head attention layer with any constantly many heads with multiple single-head attention layers.

Quoting Toyota maybe search fo the original sou

Algorithm 1 Causal Self-Attention, ATTN

Require: Parameter $\theta_{\text{ATTN}} = (W_Q, W_K, W_V, W_O)$, Input embedding $h = (h_1, \dots, h_n) \in \mathbb{R}^d \times \cdots \times \mathbb{R}^d$.

Ensure: Output embedding $h' = (h'_1, \dots, h'_n) := \text{ATTN}_{\theta_{\text{ATTN}}}(h_1, \dots, h_n)$.

$q_i := W_Q h_i,\ k_i := W_K h_i,\ v_i := W_V h_i,\ \forall i \in [n]$

$s_i := \text{softmax}(\langle q_i, k_1\rangle, \dots, \langle q_i, k_i\rangle)\|(0, \dots, 0)$.

$h'_i := W_O \sum_{j=1}^n (s_i)_j v_j$.

Feed Forward Network: It's a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\text{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d\times d} \times \mathbb{R}^d \times \mathbb{R}^{d\times d} \times \mathbb{R}^d$, it's defined as $\text{FF}_{theta_{\text{FF}}}(h) := W_2\ \text{softmax}(W_1 h + b1) + b2$, where $\text{softmax} : \mathbb{R}^d \to \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

Output Layer: It's a mapping from $\mathbb{R}^d$ to a probability distribution over Σ. Given

$\theta_{\text{OUTPUT}} \in \mathbb{R}^{|\Sigma| \times d}$, it's defined as $\text{OUTPUT}_{\theta_{\text{OUTPUT}}}(h) := \text{softmax}(\theta_{\text{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

Algorithm 2 Distribution generator

Require: Transformer parameter $\theta = (\theta_{\text{TE}}, \theta_{\text{PE}}, \{\theta^l_{\text{ATTN}}, \theta^l_{\text{FF}}\}_{l=0}^{L-1}, \theta_{\text{OUTPUT}})$ and tokens of the tape $x = (x_1, \ldots, x_n) \in \Sigma^n$.

Ensure: Output distribution $p_\theta(\cdot | x_1, \ldots, x_n)$.

1: $h_i^{(0)} \leftarrow \theta_{\text{TE}}(x_i) + \theta_{\text{PE}}(i), \ \forall i \in [n]$

2: **for** $l = 0, \ldots, L-1$ **do**

3: $\quad (h_1^{(l+0,5)}, \ldots, h_n^{(l+0,5)}) \leftarrow (h_1^{(l)}, \ldots, h_n^{(l)}) + \text{ATTN}_{\theta^{(l)}_{\text{ATTN}}}(h_1^{(l)}, \ldots, h_n^{(l)})$

4: $\quad h_i^{(l+1)} \leftarrow h_i^{(l+0,5)} + \text{FF}_{\theta^{(l)}_{\text{FF}}}(h_i^{(l+0,5)}), \ \forall i \in [n]$

5: **end for**

6: $p_\theta(\cdot | x_1, \ldots, x_i) \leftarrow \text{OUTPUT}_{\theta_{\text{OUTPUT}}}(h^{(L)})$